- Chapter V: for the fearless — eliminating cross cutting concerns from the authorization layer with NGINX
  - What is NGINX
  - Using NGINX as an application gateway
    - Adding NGINX to our Docker Compose file
    - Configuring NGINX
    - Implementing authorization logic with NGINX and NJS

Chapter V: for the fearless — eliminating cross cutting concerns from the authorization layer with NGINX

The reference for this chapter is oauth2-spring-boot/keycloak-gateway, you can use the collection named `oauth2-spring-boot-auth.json` if you want to follow along with Postman.

We left chapter IV with some questions about creating a layer of authentication that filters requests before they hit microservices: to answer such questions, we will leverage NGINX's powerful and extensible set of functionalities.

What is NGINX

NGINX is a powerful and popular open-source software used primarily as a web server, but it can also serve other roles such as a reverse proxy, load balancer and more.
What makes NGINX special is its efficiency. Traditional web servers like Apache handle each request with a separate process, which can get slow when there are a lot of visitors. NGINX, on the other hand, uses an event-driven, asynchronous architecture, which means it can handle many more requests simultaneously without using a lot of resources.
You can learn more about how an event-driven web server works by clicking here and here.

In addition to serving web pages, NGINX can also be used to:

- Reverse Proxy: Forward client requests to another server, often used to distribute load or protect backend servers;
- Load Balancer: Distribute traffic across multiple servers to ensure no single server gets overwhelmed;
- Cache: Store copies of web pages temporarily to serve them quickly without hitting the backend server every time;
- Application gateway: since NGINX can be used as a load balancer, the team behind it thought about making it extensible with custom behaviour, to accomodate the most disparate needs. Custom code can be added to its base functionality to support advanced use cases, making NGINX a versatile application gateway.

NGINX is widely used because it's fast, flexible, and can handle a lot of traffic with minimal hardware. If you're setting up a web application or service, NGINX is a key tool you might use to ensure it runs smoothly and efficiently.

Using NGINX as an application gateway

An application gateway is the component responsible for routing traffic coming from outside the system (i.e. from end users) to the right microservices. You can think of an application gateway to be some sort of "gatekeeper" between your microservice architecture and the outside world: it establishes rules to evaluate if a request should be allowed into the system or not and, when it is actually allowed, routes such request inside our system. In other words, an application gateway is the entry point to our system and, because of this, it has to be correctly scaled to fit the size of our system, when dealing with production clusters, to avoid single points of failure.
In our examples, though, we will be dealing with only one instance of NGINX configured as an application gateway for simplicity's sake, but the same configuration can be effortlessly deployed to multiple instances.

Adding NGINX to our Docker Compose file

Let's add a new service to our `compose.yml` file:

```
services:
    # grades, students, keycloak, databases...
    proxy:
        image: nginx:mainline
        networks:
            - microservices-net
        ports:
            - 8089:14000
        depends_on:
            - students
            - grades
```

We added a new service called `proxy`, using the mainline `nginx` image (the most recent one, basically). For obvious reasons, this container should be in the same network as the other microservices, since we will need to route traffic from the outside to the actual microservices.
Next, we bound NGINX to the local port `8089` and traffic from that port will be forwarded to the container's internal port `14000`.
Lastly, we added two dependencies, `students` and `grades`: this way, NGINX will start *after* these two microservices, we will see in a bit why.

Configuring NGINX

Let's add the configuration needed for routing traffic from the outside world to our microservices. First, let's create a file called `nginx/gateway.conf`:

```
server {
    listen 14000;
    http2 on;
    resolver 127.0.0.11 ipv6 off;
    # more stuff coming soon
}
```

This is a very basic configuration file for NGINX:

- The `server` block specifies that we are writing a configuration for a specific domain or host. This concept is called *virtualhosting*, which is basically the capability of a web server to handle many domains at once. NGINX implements this functionality by splitting each domain's configuration inside different files and each file will refer to a domain. Generally, a domain is referred with the `server_name` directive so, for example, if we wanted to write a configuration for the `macca.cloud` domain we will have to write `server_name macca.cloud` inside our server block. Omitting the `server_name` directive, like we did in our configuration, will result in NGINX using the server block as a "catch-all" handler, meaning that any request that does not match a particular domain will be forwarded there. Since we are not using domains, because this is an internal proxy, and because we only have one configuration, we can omit the `server_name` directive;
- With the `listen` directive we are binding a certain configuration to a certain TCP port, in this case `14000`, which is completely arbitrary. Notice that this is the same port we are using inside the `compose.yml` file;
- We turn on HTTP/2.0 using the `http2 on` directive, for more efficient traffic handling;
- We set the name resolver (DNS) to `127.0.0.11`: this is **very important**, because that address is Docker's nameserver, which gives us the ability to address containers using only their name and not their IPs, a feature that is very convenient for the piece of configuration that we will see in a second.

What we need to do now is to make our microservices reachable from the outside using NGINX. Up until now we used Docker's port forwarding to expose our microservices: this works fine for basic cases, but for more advanced production scenarios, having microservices directly exposed does not scale well, because we would have to manually map ports every time we create a new microservice. A way more maintainable mode of doing this is exposing only the application gateway to the outside by mapping its port to a host port and let the traffic flow through it.
Let's expose the `students` microservice using the concept of `location`s:

```

server{
    listen 14000;
    # http2, ...

    location /students/ {

        proxy_pass http://students:8080/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
    }

    # ... more stuff
}
```

- Using the `location` directive gives us the possibility of creating a new request route towards users can perform requests. In this example we created the `/students/` location, which essentialy exposes a path at the address `http://localhost:14000/students/*`, meaning that this block will catch all the requests that start with `/students/`. Note the **trailing slash** after the location name (the last slash): if we hadn't added such character, (i.e. if we had written `location /students { ... }`) NGINX would have created a record for the `/students` path only, not catching all the other combined paths (`/students/*`);

- Inside the `location` block we write what happens to requests that satisfy that location (i.e. any request that starts with `/students/`):
  - `proxy_pass` sets the address we want to forward the request to. The trailing slash, here, means that we want to pass the full URL path. For example, if we issued a request for `/students/grades`, NGINX would perform a request to `http://students:8080/grades` (and NOT `http://students:8080/students/grades` !). The first thing to notice is that we are using the same name we are using inside the `compose.yml` (same `service` name) file to address the `students` microservice; that's how Docker works: you can reach other containers by using their names only, not their IPs, if you are using the correct DNS resolver. This also underlines the importance of setting the `resolver` directive to the value we saw before.
    The second thing that stands out is that we are using the **internal** port (`8080`) of the `students` microservice, not the **external** one (`7777`): this is what we meant when we talked about letting traffic flow through the application gateway only; port `8080` is not reacheable from applications that are not in the same Docker network as the `students` microservice, but because NGINX is, traffic can flow from the application gateway to the container.
    The third thing that we need to understand is that it's crucial that containers that NGINX addresses (in this case `students`) are **already online** when NGINX boots the configuration, because otherwise the resolver would fail to lookup the names of the containers (because they wouldn't be online yet!). That's why we made NGINX start after `students` and `grades` in the `compose.yml` file;
  - `proxy_http_version` sets the HTTP version we want to use when forwarding requests.
  - `proxy_set_header` adds headers when forwarding requests to addresses. In this case we are passing the `Upgrade`, `Host`, `X-Forwarded-For` with their respective values, `$http_upgrade`, `$remote_addr` and `$host`, which are global NGINX variables that is generally good practice to `proxy_pass`.

Let's add another location and complete our configuration:

```

server {
    listen 14000;
    http2 on;
    resolver 127.0.0.11 ipv6=off; # docker resolver

    location /students/ {

        proxy_pass http://students:8080/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
    }

    location /grades/ {

        proxy_pass http://grades:8080/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
    }

}
```

Fantastic! We can now update our `compose.yml` and remove all the external ports from our microservices. We also have to make NGINX read the configuration we just created: this is done by placing the `nginx/gateway.conf` file in the `/etc/nginx/conf.d` folder inside NGINX's container.

```
# more stuff ...
services:
  students:
    depends_on:
      students_pg:
        condition: service_healthy
    networks:
      - microservices-net
    # no "ports" property!
    build:
      context: microservices/students
      dockerfile: Dockerfile
    mem_limit: 512m
    environment:
      - SPRING_PROFILES_ACTIVE=...
      - DB_CONNECTION_STRING=...

  grades:
    depends_on:
      grades_pg:
        condition: service_healthy
    networks:
      - microservices-net
    # no "ports" property!
    build:
      context: microservices/grades
      dockerfile: Dockerfile
    mem_limit: 512m
    environment:
      - SPRING_PROFILES_ACTIVE=...
      - DB_CONNECTION_STRING=...

    proxy:
        image: nginx:mainline
        networks:
            - microservices-net
        volumes:
            - ./nginx/gateway.conf:/etc/nginx/conf.d/gateway.conf
        ports:
            - 8089:14000
        depends_on:
            - students
            - grades

# more stuff ...
```

Implementing authorization logic with NGINX and NJS

Now that we configured NGINX to reach our microservices, it's time to inject the authorization logic inside the `location` blocks, because we want every request made to our microservices to be authenticated!
To do so, we want NGINX to be able to read the `Authorization` header of our requests. Let's add the following code at the top of our `nginx/gateway.conf` file:

```
# this extracts the authorization header from the request
map $http_authorization $header_token {
    "~*^Bearer (.*)$" $1;
    default $http_authorization;
}

# ...
```

The `map` directive, in NGINX, essentialy creates a variable, in this case `$header_token` from another, in this case `$http_authorization` (which is a global, pre-defined, NGINX variable). Inside the `map` directive we write a regex that extracts the content from the `$http_authorization` variable and places it inside the variable we are creating (`$header_token`).
If you know your fair share about regexes you already understood what we are doing: we are taking the string that comes after `Bearer` inside the `Authorization` header (so we are essentialy getting the JWT that - allegedly - comes with every request) and we are returning it using `$1`, which is the first match of the regex. If no matches are found, a default value is returned, which is, again, `$http_authorization`.
This step allowed us to introduce a global variable inside our NGINX configuration file that can be accessed from anywhere within the `server` and location blocks.

Now that we extracted the token from our authorization header, we need to read the information it holds.
To do so, we can leverage NGINX's NJS module, which is essentially a JavaScript runtime built in NGINX's core. This greatly simplifies the process of adding custom logic to configuration files: before this feature was introduced, a C module needed to be developed, compiled and linked inside NGINX's binary.
Let's create a JavaScript file called `nginx/js/check_email.js` with the following content:

```
const checkToken = (r) => {

    const access_token = r.variables.header_token;
    // we know that a jwt is composed of three parts:
    //     HEADER.PAYLOAD.SIGNATURE
    // since the contents of a jwt are NOT encrypted, we can just take the payload, base64 decode it and
read
    // what's inside!


    const split = access_token.split(".");
    if(split.length < 3){
        r.return(401);
        return;
    }

    const decoded_payload = Buffer.from(split[1], 'base64').toString();
    try{
        const json_payload = JSON.parse(decoded_payload);
        if(json_payload["client_id"] === "aggregator"){
            r.return(204);
            return;
        }
        if(!json_payload["email_verified"]){
            r.return(403);
            return;
        }
        r.return(204);
    }catch(err){
        r.return(401);
    }


}

export default {
    checkToken
}
```

This file implements essentialy the same code we wrote in Java in the previous chapter:

- we access the JWT by using `r.variables.header_token`. The paramter `r` is something that NGINX automatically passes to each JavaScript file and represent the current request being processed. Every request holds a `variables` property that contains every user-created variable: in this case we are accessing the `header_token` variable we created before when we extracted the token from the authorization header;
- next, we base64 decode the token, and:
  - if the `client_id` claim inside the token is `aggregator`, then we return a `204` HTTP status code, we will see in a bit why;
  - otherwise we check for the `email_verified` claim. If such claim is false, then we return a `403` HTTP status code, otherwise we return a `204` HTTP code (we will see, again, why, in a bit);
- lastly, we export the function as we would normaly do in any JavaScript file.

To make NGINX use this custom code, we need to do some slight modifications to our NGINX instance.
Let's start by adding the following line at the top of our `nginx/gateway.conf` file:

```
js_import validator from js/check_email.js; # add this line

# ... other stuff

server {
    # ... more stuff
}
```

This essentialy imports the JavaScript file we just created under the name of `validator`, which we will use to reference such module inside our NGINX configuration.

Next, we add a new `location` block:

```
js_import validator from js/check_email.js;

# ... more stuff

server {
    # ... more stuff

    location = .validate_token  {
        internal;
        js_content validator.checkToken;
    }

    # ... more stuff
}
```

The `js_content` directive inside this location will make the JavaScript code run when the location block is referenced inside the configuration. We specify that we want to run the `checkToken` function we created by using the `validator.checkToken` syntax.
Note that this location is marked as `internal`, meaning that it won't be mapped to an external path, but it will be only available inside the configuration file.

We then add two more internal locations:

```

# ... more stuff
server {
    # ... more stuff
    location .unauthorized {
        internal;
        default_type application/json;
        add_header Content-Type "application/json";

        return 400 '{ "success": false, "error": "gateway_bad_token" }';
    }
    location .forbidden {
        internal;
        default_type application/json;
        add_header Content-Type "application/json";

        return 400 '{ "success": false, "error": "email_not_verified" }';
    }
}
```

These two location will essentialy return JSON content with the `400` HTTP status code. We will see them in action in the next bit of configuration.

Lastly, we add these lines to every `location` block we want the authorization checks to be enabled on:

```

# ... stuff
server {
    # ... stuff
    location /grades/ {

        ### add these three lines
        auth_request .validate_token; # note that .validate_token is the location where the JavaScript code
is executed
        error_page 401 = .unauthorized;
        error_page 403 = .forbidden;
        ###

        proxy_pass http://grades:8080/;
        # ... stuff
    }
    # ... stuff
}
```

Let's break down what we just did:

- we wrote our JS module and included it in our configuration under the name `validator`;
- we added the `.validate_token` location that runs our JavaScript code;
- we configured our `location` blocks to include a new directive, called `auth_request`. This is where the magic happens:
  - when NGINX encounters this directive when processing a location block, it essentialy forwards the request to the location we specify as parameter (in our case `.validate_token`). Note that such locations, as stated in the documentation, **can only return `401`, `403` and `204` HTTP codes** (that explains why we were returning such codes in the JavaScript file). `401` means that the request is unauthorized, `403` means that the request is **forbidden** (which is different from unauthorized), meaning that the presented credentials do not have sufficient permissions to access the resource (i.e. the `email_verified` claim is not `true`) and `204` means that everything is ok and the request can proceed.
    When the request hits the `.validate_token` location, the JavaScript code is run, and the whole request object is passed to our script.
  - After the script inside the `.validate_token` has completed its execution, a value from the `auth_request` will be returned, one between `204`, `401` and `403`.
  - By using the `error_page` directive, we can access the previous `auth_request` return code: if such code is `401`, then the `.unauthorized` location block will be displayed (and the JSON that comes with it), effectively terminating the request on the spot. Similar fate have the requests that return `403`, of course, which will be redirected to the `.forbidden` location block. If `204` is returned, the request is good to go and can be forwarded to the corresponding microservice.

Our complete `nginx/gateway.conf` file looks like this:

```
js_import validator from js/check_email.js;

# this extracts the authorization header from the request
map $http_authorization $header_token {
    "~*^Bearer (.*)$" $1;
    default $http_authorization;
}


server {
    listen 14000;
    http2 on;
    resolver 127.0.0.11 ipv6=off; # docker resolver
    include mime.types;

    location = .validate_token  {
        internal;
        js_content validator.checkToken;
    }

    location /students/ {

        # we want to validate the token!
        auth_request .validate_token;
        error_page 401 = .unauthorized;
        error_page 403 = .forbidden;

        proxy_pass http://students:8080/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header Authorization "Bearer $header_token";
        proxy_cache_bypass $http_upgrade;
    }

    location /grades/ {

        # we want to validate the token!
        auth_request .validate_token;
        error_page 401 = .unauthorized;
        error_page 403 = .forbidden;

        proxy_pass http://grades:8080/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header Authorization "Bearer $header_token";
        proxy_cache_bypass $http_upgrade;
    }

    location .unauthorized {
        internal;
        default_type application/json;
        add_header Content-Type "application/json";

        return 400 '{ "success": false, "error": "gateway_bad_token" }';
    }
    location .forbidden {
        internal;
        default_type application/json;
        add_header Content-Type "application/json";

        return 400 '{ "success": false, "error": "email_not_verified" }';
    }

}
```

The resulting behaviour will be that if we issue requests that have tokens that are not well-encoded, a JSON `400` response with the following contents will be presented:

```
 "success": false, "error": "gateway_bad_token" }
```

Whereas if we issue a request with a valid token containing the `email_verified` claim set to `false` or `undefined`, we get:

```
{ "success": false, "error": "email_not_verified" }
```

In both these cases, the request **never reaches the microservices**, which is the behaviour we expect from our system.

At this stage we can remove the configuration bean we created in the previous chapter, `OAuth2Configuration.java`, because the logic is baked inside the gateway now.
Important note: with this configuration, the gateway **does not verify the authenticity of the token**! This is still the microservices' job, so Spring's Web Security module must still be enabled in our microservices to make sure they verify the authenticity of the token.
Naturally, that responsibility can be, too, lifted from the microservices (and it's good practice to do so, to reduce their sizes), but for simplicity's sake I chose not to include cryptographical checks in this chapter.

Next chapter: Appendix: refreshing access tokens
Previous chapter: Chapter IV: implementing authentication